
REST and RESTful API

Lâm Hoài Bảo (lhbao@ctu.edu.vn)

Agenda

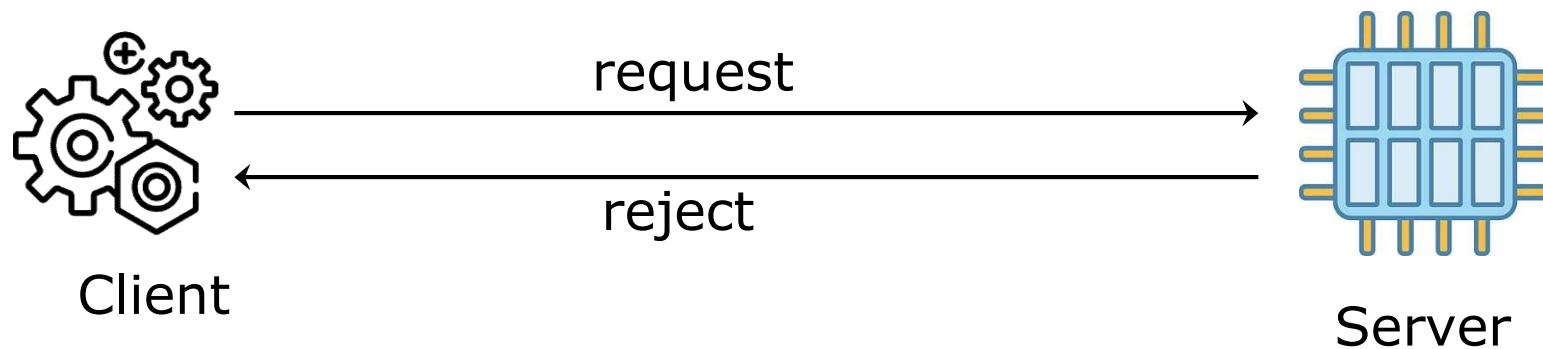
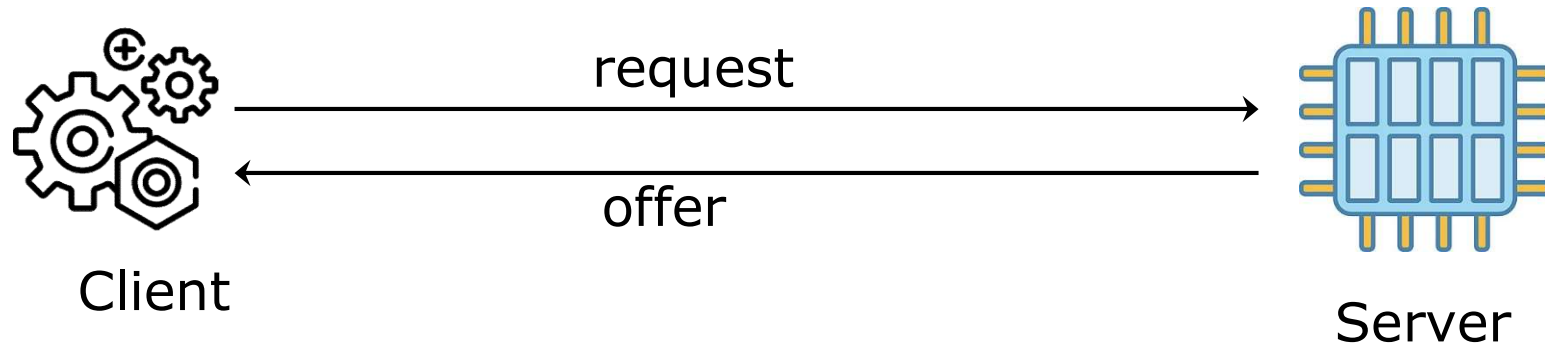
- Representational State Transfer (REST)
- Client Server architectures
- REST API
- Summary

REpresentational State Transfer (REST)

- Architectural style for distributed hypermedia systems (Roy Fielding 2000).
 - Provide software engineering principles and constraints
- A Web API (Web Service) coinciding the REST architectural style: REST API (RESTful API)

Client Server

- A server component: listen for requests
- A client component: send a request to the server via a connector



Client Server

- Separation of concerns: client and server evolve independently
 - Separating UI (client) from data storages (server)
 - Portability of UI across multiple platforms
 - Improve scalability by simplifying the server components
-

Stateless

■ Client server communication: stateless

- Request from client to server must contain all necessary information
- Session state are kept on the client

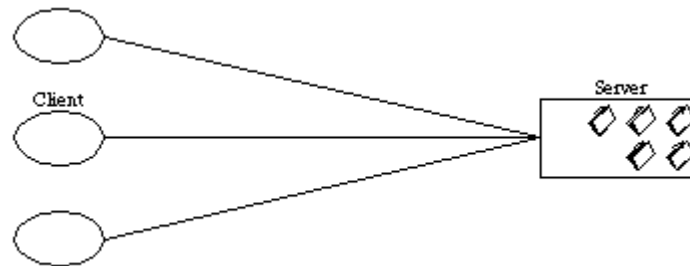


Figure 5-3. Client-Stateless-Server

■ Trade-offs

- Network performance: repetitive data sent in a series of requests
- Server's control over consistent application behavior

Cache

- Data within a response to request must be labeled as cacheable or non-cacheable

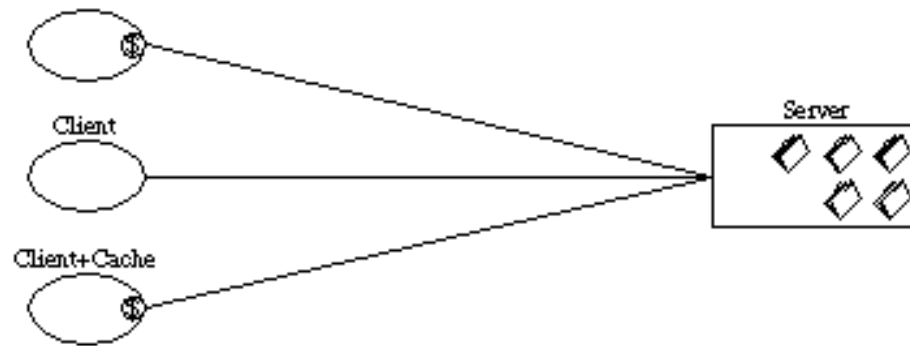


Figure 5-4. Client-Cache-Stateless-Server

- Cache: reuse the response data for later
- Trade-offs:
 - Decrease of reliability

Uniform interface

- Emphasis on user interface between components
- Constraints
 - Identification of resources: the interface must uniquely identify each resource involved in the interaction
 - Manipulation of resources through representations: the resource should have uniform representations in the server response
 - Self-descriptive messages: resource representation should carry enough information to describe how to process the message
 - Hypermedia as the engine of application state: the client should have only the initial URI of the application

Layer system

- Hierarchical layers by constraining component behavior.
- Example: MVC pattern

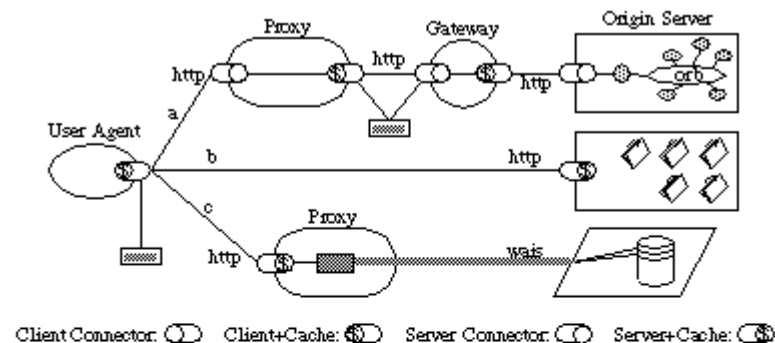
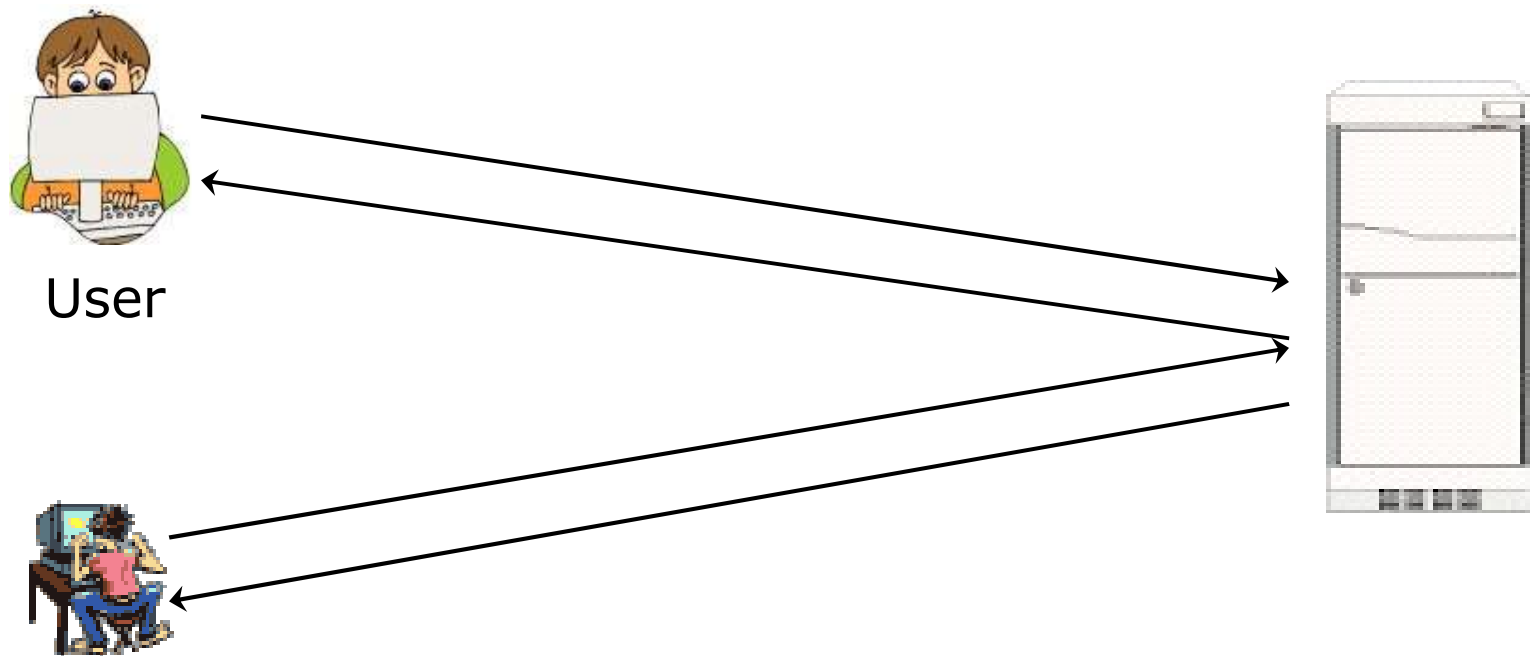


Figure 5-10. Process View of a REST-based Architecture

A user agent is portrayed in the midst of three parallel interactions: a, b, and c. The interactions were not satisfied by the user agent's client connector cache, so each request has been routed to the resource origin according to the properties of each resource identifier and the configuration of the client connector. Request (a) has been sent to a local proxy, which in turn accesses a caching gateway found by DNS lookup, which forwards the request on to be satisfied by an origin server whose internal resources are defined by an encapsulated object request broker architecture. Request (b) is sent directly to an origin server, which is able to satisfy the request from its own cache. Request (c) is sent to a proxy that is capable of directly accessing WAIS, an information service that is separate from the Web architecture, and translating the WAIS response into a format recognized by the generic connector interface. Each component is only aware of the interaction with their own client or server connectors; the overall process topology is an artifact of our view.

Code on demand

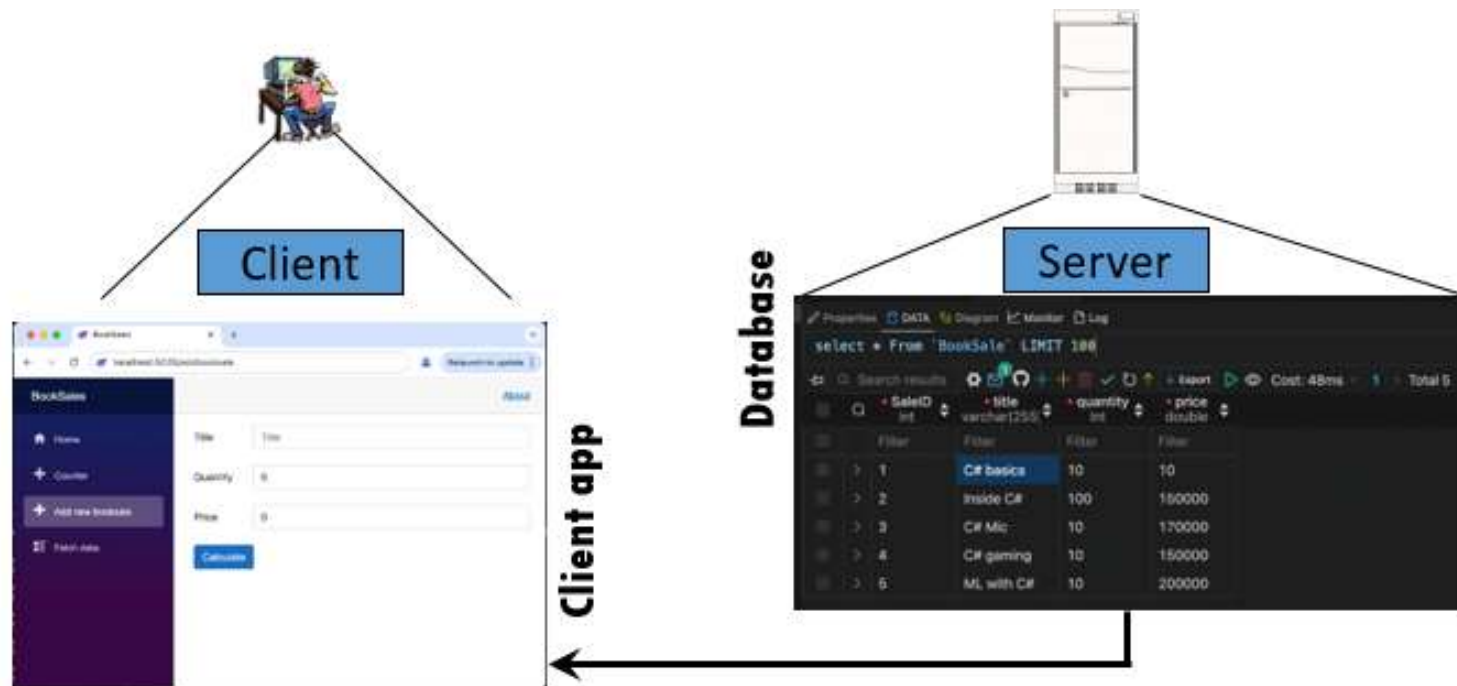
- Client functionality to extend by downloading and executing code



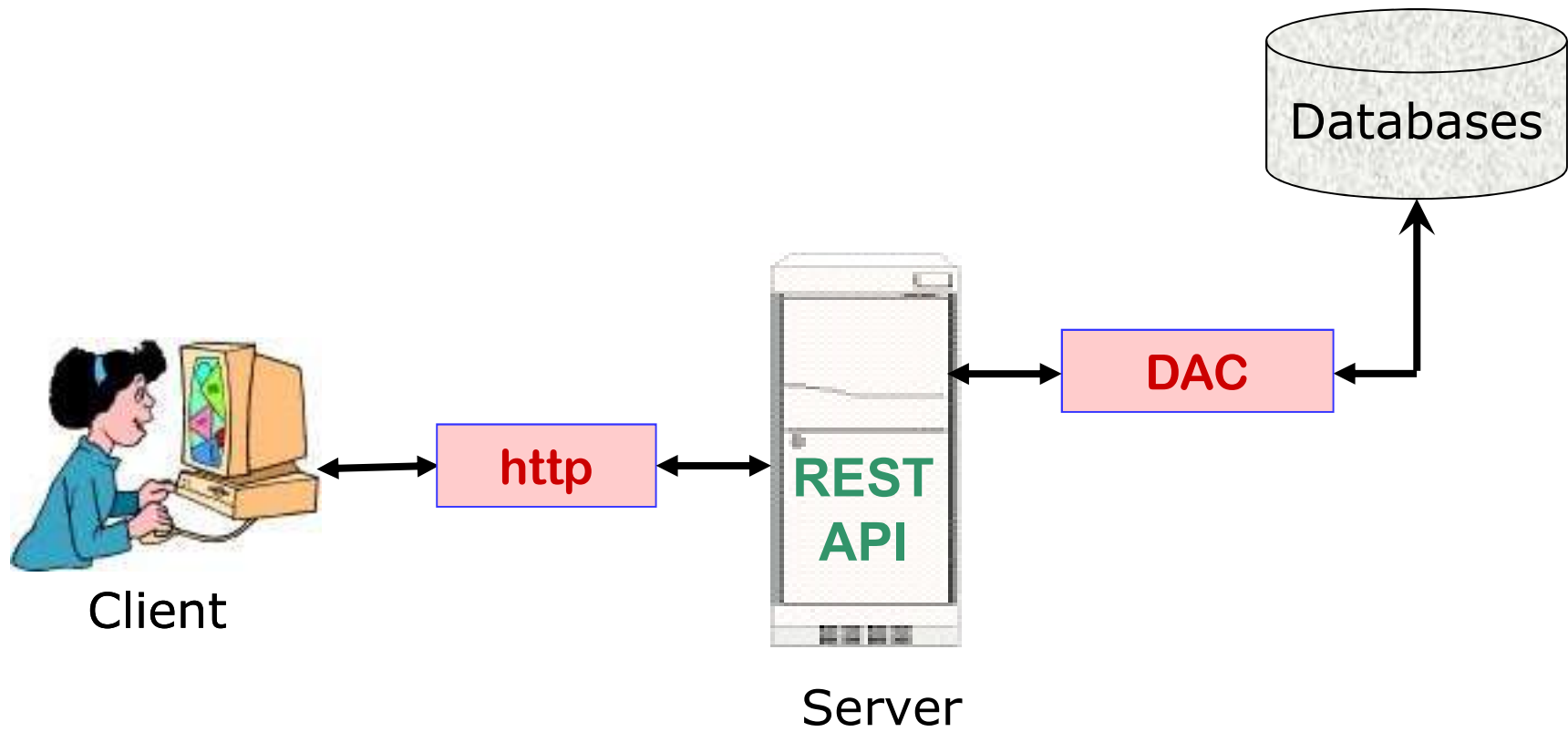
Agenda

- Representational State Transfer (REST)
- Client Server architectures
- RESTful API
- Summary

A simple client server architecture



RESTful API client server architecture



Agenda

- Representational State Transfer (REST)
- Client Server architectures
- RESTful API
- Summary

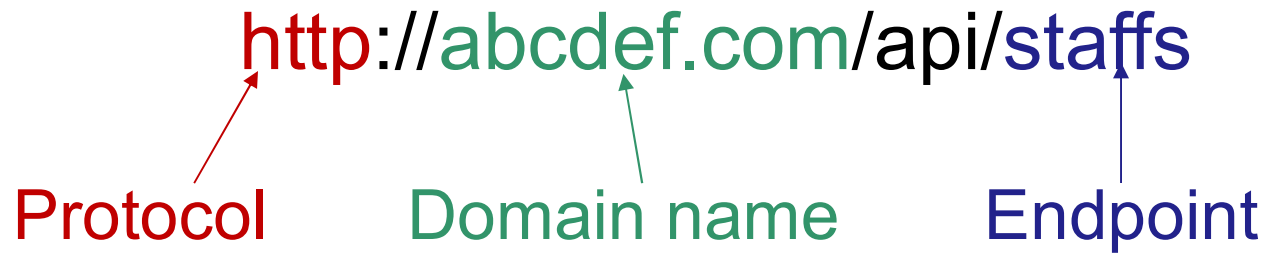
What is RESTful API?

- An interface between 2 computers use to exchange information securely over the internet
- Example: Get data from a third party: temperature, humidity, wind speed
- An Application Programming Language (API): rules must follow to communicate with other software systems
- Web API
 - Clients: users want to access information from the web
 - Resources: information that different applications provide to clients

Resource

http://abcdef.com/api/staffs

Protocol Domain name Endpoint

A diagram showing the components of a Uniform Resource Identifier (URI). The URI 'http://abcdef.com/api/staffs' is displayed with three parts highlighted in different colors: 'http' in red, 'abcdef.com' in green, and '/api/staffs' in blue. Below each colored part is a label: 'Protocol' in red, 'Domain name' in green, and 'Endpoint' in blue. Arrows point from each label to its corresponding part of the URI.

- Each resource is identified by an URI (Uniform Resource Identifier)
- Method: send http verb to the server
 - GET, POST, PUT, DELETE

Common operations

HTTP Verb	CRUD
Post	Create
Get	Read
Put	Update
Delete	Delete

URI + Http Verb

URI	Http Verb	Description
http://abcdef.com/api/staffs	Get	Get all staffs
http://abcdef.com/api/staffs/1	Get	Get a staff with id = 1
http://abcdef.com/api/staffs/1	Put	Update a staff with id = 1
http://abcdef.com/api/staffs	Post	Create a new staff
http://abcdef.com/api/staffs/1	Delete	Delete a staff with id = 1

Agenda

- Representational State Transfer (REST)
- Client Server architectures
- REST API
- Summary

Summary

- A Web API conforming to the REST architectural style is called REST API (RESTful API)
- REST assists architectural design of a system that encompasses multiple organizational boundaries

Q&A
